

13 · Despliegue y operación

Documentacion del sistema

Version analizada: **0.1.0**

Commit analizado: **5e91229**

Fecha de generacion: **2026-08-27**

Generado a partir de 13-deployment-and-operations.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Entornos
2. Proceso de construcción
3. Empaquetado
4. Release en un comando
5. Integración y despliegue continuos
6. Instalación en el equipo del usuario
7. Migraciones
8. Operación diaria
9. Logs, métricas y monitoreo
10. Respaldo y recuperación
11. Rollback
12. Mantenimiento
13. Generación de la documentación en PDF
14. Procedimiento de release paso a paso

1. Entornos

El producto **no tiene entornos** en el sentido habitual: no hay servidor, ni base de datos remota, ni configuración por entorno. Lo que sí hay son cuatro contextos de ejecución:

Contexto	Qué es	Cómo se obtiene
Desarrollo	Binario compilado localmente	<code>cargo build</code>
Validación	Ejecución en la CI de GitHub	Automática en cada push a <code>main</code>
Distribución	<code>.app</code> y <code>.dmg</code> firmados por nadie	<code>scripts/package-*.sh</code> o la release
Uso	El Mac del usuario final	Descarga o compilación propia

2. Proceso de construcción

Diagrama Mermaid #1 — código fuente (no renderizado en PDF)

```
flowchart LR
    SRC["src/*.rs"] --> CARGO["cargo build --release"]
    CARGO --> BIN["target/release/rootcause"]
    SRC --> CARGOU["cargo build --release --target aarch64-apple-darwin<br/>+ x86_64-apple-darwin"]
    CARGOU --> LIPO["lipo -create"]
    LIPO --> UNI["binario universal"]
    BIN --> PKG["package-app.sh"]
    UNI --> PKG
    ICON["assets/rootcause-icon.svg"] --> MKICON["make-icon.py + iconutil"]
    MKICON --> ICNS["AppIcon.icns"]
    PLIST["packaging/macos/Info.plist"] --> PKG
    ICNS --> PKG
    PKG --> APP["dist/RootCause.app"]
    APP --> DMG["package-dmg.sh + hdiutil"]
    DMG --> IMG["dist/RootCause-0.1.0.dmg<br/>dist/SHA256SUMS.txt"]
```

2.1 Perfil de release

```
[profile.release]
opt-level = 3
lto = true
codegen-units = 1
strip = true
```

`lto` y `codegen-units = 1` alargan la compilación a cambio de un binario más pequeño y rápido; `strip` elimina los símbolos de depuración, lo que reduce el tamaño y también la información que un binario distribuido revela sobre el código.

2.2 Binario universal

`scripts/package-app.sh --universal` compila para `aarch64-apple-darwin` y `x86_64-apple-darwin` y los une con `lipo`. Requiere `rustup` (Homebrew solo instala el target del host). **Si no es posible, no falla:** avisa y se degrada al binario nativo. La CI sí construye siempre el universal y lo verifica con `lipo -info`.

3. Empaquetado

3.1 El `.app`

`scripts/package-app.sh` construye la estructura estándar del bundle:

```
dist/RootCause.app/
  Contents/
    Info.plist ← packaging/macos/Info.plist con __VERSION__ sustituido
    MacOS/rootcause ← binario
    Resources/AppIcon.icns
```

`Info.plist` declara `LSMinimumSystemVersion = 13.0`, el identificador `dev.vladimiracuna.rootcause`, `NSHighResolutionCapable` y `NSAppleEventsUsageDescription` con el texto que verá el usuario cuando se consulten los login items.

3.2 El `.dmg`

`scripts/package-dmg.sh` monta un escenario con el `.app`, un enlace a `/Aplicaciones` y un `LÉEME.txt` que explica el aviso de Gatekeeper, y crea la imagen con `hdiutil`.

Detalle operativo importante: `hdiutil` solo crea imágenes sobre APFS o HFS+. Si el repositorio vive en un volumen exFAT o de red —habitual en discos externos—, falla con «Operación no permitida». El script lo evita creando la imagen en un directorio temporal del disco interno y copiando después el `.dmg`, que es solo un archivo, al destino final.

4. Release en un comando

```
./scripts/release-product.sh # construye dist/ y para ahí
./scripts/release-product.sh --verify-environment # verifica el entorno primero
./scripts/release-product.sh --publish # además etiqueta y publica
./scripts/release-product.sh --publish --watch # y espera al workflow
./scripts/release-product.sh --publish --tag-only # empuja el tag; construye la CI
./scripts/release-product.sh --skip-checks # salta fmt/clippy/tests
```

Siete fases, en este orden:

Fase	Qué hace	Puede abortar
1 · Entorno	<code>verify-environment</code> <code>.sh</code> si se pidió	Sí
2 · Validación	<code>fmt</code> , <code>clippy</code> , <code>test</code> , ambas ediciones	Sí
3 · Empaquetado	<code>.app</code> universal, <code>.dmg</code> , <code>.zip</code> , hashes	Sí
4 · Verificación	Que los artefactos existan, no estén vacíos, el binario sea ejecutable y universal, y que las versiones coincidan	Sí
5 · Notas	Genera el cuerpo de la release	No
6 · Resumen	Lista lo construido	No
7 · Publicación	Solo con <code>--publish</code>	Sí

La fase 4 comprueba tres coincidencias que suelen fallar en las releases hechas a mano:

1. `lipo -info` contiene `arm64` y `x86_64`.
2. `rootcause --version` coincide con la versión de `Cargo.toml`.
3. `CFBundleShortVersionString` del `Info.plist` coincide también.

El comentario del script explica por qué: *un artefacto vacío o ausente que llega a una release es peor que un fallo de build: nadie lo descubre hasta que alguien intenta usarlo.*

La fase 7 comprueba seis condiciones **antes de tocar el remoto**: binario universal (salvo `--tag-only`), `gh` instalado, `gh` autenticado, rama `main`, árbol de trabajo limpio, etiqueta libre y ausencia de una release previa con esa etiqueta.

5. Integración y despliegue continuos

5.1 ci.yml

Disparo: push y pull request a `main`, o manual. Detalle en [12 · Pruebas y calidad](#). Publica como artefacto el binario de release junto con `README.md`, `LICENSE`, `SECURITY.md`, `docs/**` y `scripts/**`.

5.2 release-macos.yml

Disparo: etiquetas `v*` o manual. Permisos `contents: write`.

1. Toolchain con ambos targets y caché.
2. Valida (`fmt`, `clippy`, `test`).
3. `./scripts/package-app.sh --universal`.
4. **Verifica que el binario es universal** con `lipo -info`.
5. `./scripts/package-dmg.sh`.
6. Comprime el `.app` y añade su hash a `SHA256SUMS.txt`.
7. Lista los artefactos y los sube como artefacto del workflow.
8. Con etiqueta `v*`, crea la release de GitHub con notas generadas y el cuerpo fijo que advierte de la falta de firma y notarización.

5.3 deploy-landing.yml

Disparo: push a `main` que toque `landing/**` o el propio workflow. Publica esa carpeta en GitHub Pages con concurrencia `pages` y `cancel-in-progress: false`.

6. Instalación en el equipo del usuario

Vía	Pasos	Nota sobre Gatekeeper
Compilar (recomendada)	<code>cargo build --release</code>	No aplica: el binario es local
<code>.dmg</code>	Abrir, arrastrar a Aplicaciones	Primera apertura: autorizar en Ajustes → Privacidad y seguridad
<code>.zip</code>	Descomprimir, mover a Aplicaciones	Igual
Homebrew	No disponible: el cask es una plantilla sin tap	—

Los artefactos **no están firmados ni notarizados**, y el cuerpo de la release lo advierte explícitamente, recomendando compilar desde el código como vía preferente.

7. Migraciones

No hay sistema de migraciones. El esquema se crea con `CREATE TABLE IF NOT EXISTS` en cada arranque, lo que tiene una consecuencia operativa concreta:

Cambio de esquema	Qué pasa hoy
Añadir una tabla	Se crea sola en el siguiente arranque
Añadir una columna a una tabla existente	No se aplica: la tabla ya existe y no se altera
Cambiar un tipo	Idem

Es decir: cualquier evolución de las tablas existentes exigirá añadir lógica de migración o documentar el borrado del archivo.
Registrado en [15 · Riesgos](#).

8. Operación diaria

8.1 Rutina recomendada

1. Abrir la aplicación y dejarla refrescar; revisar el semáforo y las alertas.
2. En Persistencia, revisar lo marcado como NUEVA o MODIFICADA. Si se reconoce, aceptar la baseline; si no, investigar antes de aceptar.
3. En Seguridad, comprobar que los seis controles están en el estado esperado.
4. En Privacidad, revisar los permisos sensibles concedidos.
5. En Red, aceptar la red conocida solo cuando todos los equipos sean identificables.
6. Generar un reporte (■R) cuando haya algo que documentar.

La aceptación de baselines es la operación más delicada del producto: aceptar sin revisar convierte un hallazgo en estado normal, y a partir de ahí deja de reportarse.

8.2 Equivalente en consola

```
rootcause status           # revisión rápida
rootcause persistence      # cambios vs baseline
rootcause persistence --accept # aceptar tras revisar
rootcause security --accept
rootcause tcc --accept
rootcause network --accept
rootcause report           # evidencia en Markdown
```

9. Logs, métricas y monitoreo

Aspecto	Estado
Archivo de log propio	No existe. La salida va a <code>stdout/stderr</code>
Nivel de log configurable	No
Métricas	Las que guarda la tabla <code>snapshots</code> (CPU, memoria, E/S, cachés)
Monitoreo externo	No integrado; se puede construir sobre <code>--json</code>
Alertas	Notificación del sistema ante alerta crítica
Trazas	No

La ausencia de un archivo de log es coherente con el producto —el historial SQLite cumple ese papel— pero complica diagnosticar un fallo de arranque, donde el único rastro es lo que se imprime en la terminal.

10. Respaldo y recuperación

Qué	Cómo	Frecuencia sugerida
Historial	<code>rootcause history --backup</code>	Antes de cualquier cambio importante

Todo el estado	Copiar <code>~/Library/Application Support / RootCauseInspector /</code>	Antes de actualizar
Configuración	Copiar <code>rootcause-config.json</code>	Tras ajustarla
Reportes	Ya son archivos independientes en <code>~/Documents/RootCause/reports/</code>	—

Recuperación: devolver los archivos a su sitio con la aplicación cerrada. Detalle y consecuencias en [07 · Base de datos §10](#).

11. Rollback

Situación	Cómo revertir
Versión nueva con problemas	Descargar el <code>.dmg</code> anterior de las releases, o <code>git checkout <tag></code> y recompilar
Configuración que rompe la detección	Borrar <code>rootcause-config.json</code> y ejecutar <code>config init</code>
Baseline aceptada por error	No hay deshacer. Hay que borrar el SQLite (se pierde el historial) o esperar a que el elemento cambie otra vez
Limpieza de cachés	No hay deshacer; las cachés se regeneran solas
Proceso finalizado por error	Volver a lanzarlo manualmente

La segunda fila es la más relevante para operación: **aceptar una baseline es irreversible** con las herramientas actuales. Registrado como riesgo.

12. Mantenimiento

Tarea	Frecuencia	Comando
Actualizar dependencias	Trimestral	<code>cargo update + cargo test</code>
Revisar vulnerabilidades	Mensual	<code>cargo audit</code> (no configurado todavía)
Revisar la CI	Cada release	Pestaña Actions
Regenerar la documentación en PDF	Cada cambio en <code>docs/system-documentation/</code>	<code>python3 scripts/build-docs-pdf.py</code>
Revisar el tamaño del SQLite	Semestral	<code>du -h ~/Library/Application\ Support / RootCauseInspector /</code>
Subir versión	Por release	Editar <code>Cargo.toml</code> → <code>release-product.sh</code>

13. Generación de la documentación en PDF

13.1 Requisitos

```
python3 -m pip install markdown xhtml2pdf
```

Ambas son dependencias **solo de documentación**: no forman parte del producto ni de su compilación.

13.2 Uso

```
python3 scripts/build-docs-pdf.py          # todos los documentos
python3 scripts/build-docs-pdf.py 07 11    # solo los que empiecen por 07 y 11
python3 scripts/build-docs-pdf.py --check  # solo comprueba dependencias
python3 scripts/build-docs-pdf.py --out RUTA # directorio de salida alternativo
```

Fuente única: los Markdown de `docs/system-documentation/`. El script **no los edita**: los lee, los convierte a HTML con una hoja de estilo pensada para papel y los renderiza a PDF en `docs/system-documentation/pdf/`.

13.3 Qué produce

- Un PDF por documento, con portada identificable (sistema, documento, fecha, versión).
- Índice propio en los documentos con cuatro o más encabezados de nivel 2.
- Tablas con tamaño de fuente reducido y partición de palabras largas para que no desborden.

13.4 Limitaciones conocidas, deliberadas

Limitación	Motivo	Mitigación
Los diagramas Mermaid no se rasterizan	El motor de PDF no ejecuta JavaScript	Se incluyen como bloque de código monoespaciado con un aviso: el PDF sigue siendo autocontenido y el diagrama se puede pegar en cualquier visor Mermaid
Los enlaces relativos entre documentos no navegan	El PDF es un archivo suelto	Se conservan como texto
Tablas muy anchas	Ancho de página fijo	Fuente reducida y partición de palabras

La alternativa para los diagramas —depender de Node y del CLI de Mermaid— haría el script no reproducible en un equipo limpio, que es justo lo contrario de lo que se busca en la documentación de un proyecto sin dependencias.

14. Procedimiento de release paso a paso

1. Actualizar `version` en `Cargo.toml`.
2. Actualizar la documentación que cite la versión (`README.md`, portada de esta carpeta).
3. `./scripts/ci-local.sh` → todo en verde.
4. `python3 scripts/build-docs-pdf.py` si cambió la documentación.
5. `git commit` y `git push` a `main`; esperar la CI en verde.
6. `./scripts/release-product.sh` → revisar `dist/`.
7. `./scripts/release-product.sh --publish --watch`.
8. Comprobar la release publicada y descargar el `.dmg` para verificar que abre.

`docs/RELEASE_CHECKLIST.md` mantiene la lista de verificación del producto; este documento describe la mecánica.

Siguiente lectura recomendada: [14 · Solución de problemas](#).